

DAT 5566 – Big Data and Cloud Computing 2024 – Homework #2

DAT 5566: Big Data and Cloud Computing

Fall 2025, Mini B

Homework #2

Student ID: 606379

Student Name: Yuchen Wu

Part 1: Linux Code Practice (40 pts, 10 each)

a) Please write a command to find the longest days on market for vehicles with body_type as Sedan.

Please print only the corresponding 'daysonmarket'.

```
cat used_cars.csv | grep -i "Sedan" | cut -d',' -f4 | sort -n | tail -1
```

Result:

```
1596
```

b) Please write a command to find the top 3 franchise_make in Avenel (i.e., the most frequent car brands in that city).

```
cat used_cars.csv | grep -i "Avenel" | cut -d',' -f4 | sort | uniq -c | sort -nr | head -3
```

Result

```
78 Jeep
66 Nissan
50 Toyota
```

c) Let's do a simple word count using Linux commands. Find the most 3 common words in the exterior_color column. (Reminder: find the 3 common words, not the whole line.)

```
cat used_cars.csv | cut -d',' -f7 | tr ' ' '\n' | sort | uniq -c | sort -nr | head -3
```

Result:

```
4786 Metallic
4164 Black
3619 White
```

d) Find the most frequent car with the word "leather" in its description. For this, if a row contains "leather", we consider it. Make sure to be case insensitive.

```
cat used_cars.csv | grep -i "leather" | cut -d',' -f15,18 | sort | uniq -c | sort -nr | head -1
```

Result:

```
677 Jeep,Grand Cherokee
```

Part 2: MapReduce Practice (60 pts)

In these questions, try to bring as clean as possible data into your results.

We want to analyze the maximum and average price of vehicles based on their fuel type. To do so:

- If a row contains "Electric" in the fuel_type column, classify it as "Electric".
- Otherwise, classify it as "Non-Electric".
- Ignore rows where the price is above \$200,000 (to remove outliers).

Please note that you need to run your code on HDFS. For this question, you need 3 files: mapper.py, reducer.py, and bash.sh to run the MapReduce on HDFS.

Note: If you like to have only one output file after MapReduce, add this line into your bash code (before the input line):

-Dmapred.reduce.tasks=1

i. (15 pts) Write the mapper code to find the requested summary.

Make sure to run it in debugging mode, before sending it to the HDFS.

Create the .py document

```
touch EcarMap.py
nano EcarMap.py
```

Map python function code

```
#!/usr/bin/env python3
import sys
import csv

for line in sys.stdin:
    line = line.strip()
    if not line or line.startswith("vin"):
        continue

    try:
        fields = list(csv.reader([line]))[0]
        fuel_type = fields[8].strip()
        price = float(fields[18].strip())
```

```

# Skip outliers
if price > 200000 or price <= 0:
    continue

# Classify fuel type
if "electric" in fuel_type.lower():
    category = "Electric"
else:
    category = "Non-Electric"

# Output: key \t value
print(f"{category}\t{price}")

except Exception:
    continue

```

Check Locally:

```
cat used_cars.csv | python3 mapper.py | head
```

Result:

```

[wyuchen@ip-172-31-6-2 ~]$ cat used_cars.csv | python3 EcarMap.py | head
Non-Electric      23141.0
Non-Electric      46500.0
Non-Electric      46995.0
Non-Electric      67430.0
Non-Electric      48880.0
Non-Electric      66903.0
Non-Electric      23695.0
Non-Electric      68520.0
Non-Electric      51245.0
Non-Electric      84399.0

```

ii. 30 pts) Write the reducer code to find the requested summary.

Make sure to run it in debugging mode, before sending it to the HDFS.

Create the .py document

```
touch EcarReduce.py
nano EcarReduce.py
```

Edit python reduce function

```
#!/usr/bin/env python3
import sys

current_type = None
total_price = 0
count = 0
max_price = 0

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue

    try:
        category, price = line.split('\t')
        price = float(price)
    except ValueError:
        continue

    if current_type == category:
        total_price += price
        count += 1
        if price > max_price:
            max_price = price
    else:
        if current_type:
            avg_price = total_price / count if count > 0 else 0
            print(f"{current_type}\tAverage Price: {avg_price:.2f}\tMax Price: {max_price:.2f}")
        current_type = category
        total_price = 0
        count = 0
        max_price = 0
```

```

# Reset for new category
current_type = category
total_price = price
count = 1
max_price = price

# Output the last group
if current_type:
    avg_price = total_price / count if count > 0 else 0
    print(f"{current_type}\tAverage Price: {avg_price:.2f}\tMax Price: {max_price:.2f}")

```

Check locally

```
$ cat used_cars.csv | python3 EcarMap.py | sort | python3 EcarReduce.py
```

Result

```

Electric      Average Price: 35131.25 Max Price: 82995.00
Non-Electric  Average Price: 28689.00 Max Price: 192500.00

```

iii. 10 pts) Write the bash code to run on HDFS.

Edit bash code using nano

```

#!/bin/bash
hadoop jar /opt/cloudera/hadoop-streaming-3.0.0-cdh6.2.0.jar \
-D mapred.reduce.tasks=1 \
-input used_cars.csv \
-output usedcars_output \
-file EcarMap.py \
-file EcarReduce.py \
-mapper EcarMap.py \
-reducer EcarReduce.py

```

iv. 5 pts) Which group has a higher price? Are the results as expected?

```
[wyuchen@ip-172-31-6-2 ~]$ hdfs dfs -cat usedcars_output/part-00000  
Electric      Average Price: 35131.25 Max Price: 82995.00  
Non-Electric  Average Price: 28689.00 Max Price: 192500.00
```

The Electric Group has higher price. The results are as expected.

Fuel Type	Average Price	Maximum Price
Non-Electric	28689.00	192500.0
Electric	35131.25	82995.0